



Classification Algorithm in Machine Learning





Classification Algorithm in Machine Learning

The supervised machine learning algorithm can be broadly classified into regression and classification algorithms. In regression algorithms, we have predicted the output for continuous values, but to predict the categorical values, we need classification algorithms.



What is the Classification Algorithm?

The classification algorithm is a supervised learning technique that is used to identify the category of new observations on the basis of training data. In classification, a program learns from the given dataset or observations and then classifies new observation into a number of classes or groups. Such as, **yes or no, 0 or 1, spam or not spam, cat or dog**, etc. Classes can be called as targets/labels or categories.



What is the Classification Algorithm?

Unlike regression, the output variable of Classification is a category, not a value, such as "Green or Blue", "fruit or animal", etc. Since the Classification algorithm is a Supervised learning technique, hence it takes labeled input data, which means it contains input with the corresponding output.

In classification algorithm, a discrete output function(y) is mapped to input variable(x).

$y=f(x)$, where y = categorical output

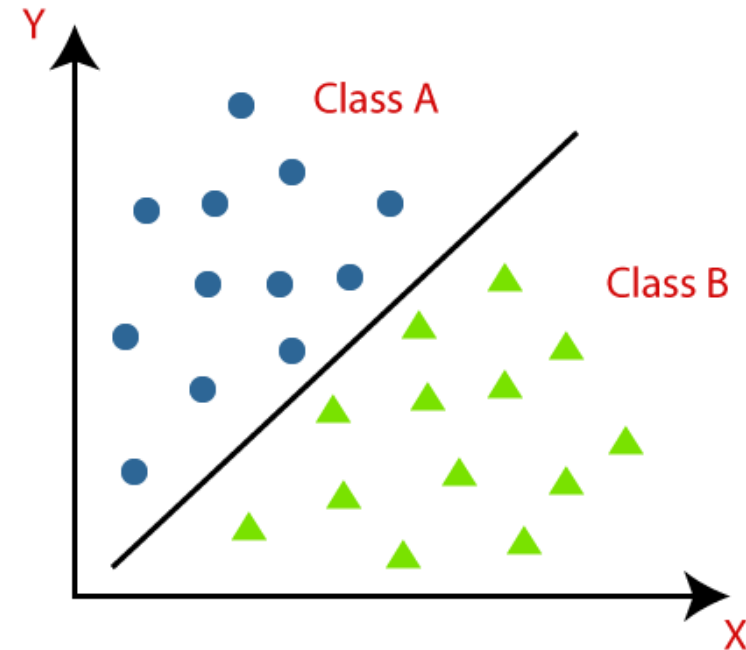
The best example of an ML classification algorithm is Email Spam Detector.

The main goal of the Classification algorithm is to identify the category of a given dataset, and these algorithms are mainly used to predict the output for the categorical data.



What is the Classification Algorithm?

Classification algorithms can be better understood using the below diagram. In the below diagram, there are two classes, class A and class B. These classes have features that are similar to each other and dissimilar to other classes.





What is the Classification Algorithm?

The algorithm which implements the classification on a dataset is known as a classifier. There are two types of classifications:

- **Binary classifier:** if the classification problem has only two possible outcomes, then it is called as binary classifier.

Examples: YES or NO, MALE or FEMALE, SPAM or NOT SPAM, CAT or DOG, etc.

- **Multi-class classifier:** if a classification problem has more than two outcomes, then it is called as multi-class classifier.

Example: classifications of types of crops, classification of types of music.



Learners in Classification Problems:

In the classification problems, there are two types of learners:

- **Lazy Learners:** Lazy Learner firstly stores the training dataset and wait until it receives the test dataset. In Lazy learner case, classification is done on the basis of the most related data stored in the training dataset. It takes less time in training but more time for predictions.

Example: K-NN algorithm, Case-based reasoning

- **Eager Learners:** Eager Learners develop a classification model based on a training dataset before receiving a test dataset. Opposite to Lazy learners, Eager learners take More time in training and less time in prediction.

Example: Decision Trees, Naïve Bayes, ANN



Types of ML Classification Algorithms:

Classification algorithms can be further divided into the mainly two category:

- **Linear models**

- Logistic regression
- Support vector machines

- **Non-linear models**

- K-nearest neighbours
- Kernel SVM
- Naïve bayes
- Decision tree classification
- Random forest classification



Difference between Regression and Classification

Regression Algorithm	Classification Algorithm
In Regression, the output variable must be of continuous nature or real value.	In Classification, the output variable must be a discrete value.
The task of the regression algorithm is to map the input value (x) with the continuous output variable(y).	The task of the classification algorithm is to map the input value(x) with the discrete output variable(y).
Regression Algorithms are used with continuous data.	Classification Algorithms are used with discrete data.
In Regression, we try to find the best fit line, which can predict the output more accurately.	In Classification, we try to find the decision boundary, which can divide the dataset into different classes.
Regression algorithms can be used to solve the regression problems such as Weather Prediction, House price prediction, etc.	Classification Algorithms can be used to solve classification problems such as Identification of spam emails, Speech Recognition, Identification of cancer cells, etc.
The regression Algorithm can be further divided into Linear and Non-linear Regression.	The Classification algorithms can be divided into Binary Classifier and Multi-class Classifier.



Logistic Regression in Machine Learning

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1.**
- Logistic Regression is much similar to the Linear Regression except that how they are used. Linear Regression is used for solving Regression problems, whereas **Logistic regression is used for solving the classification problems.**

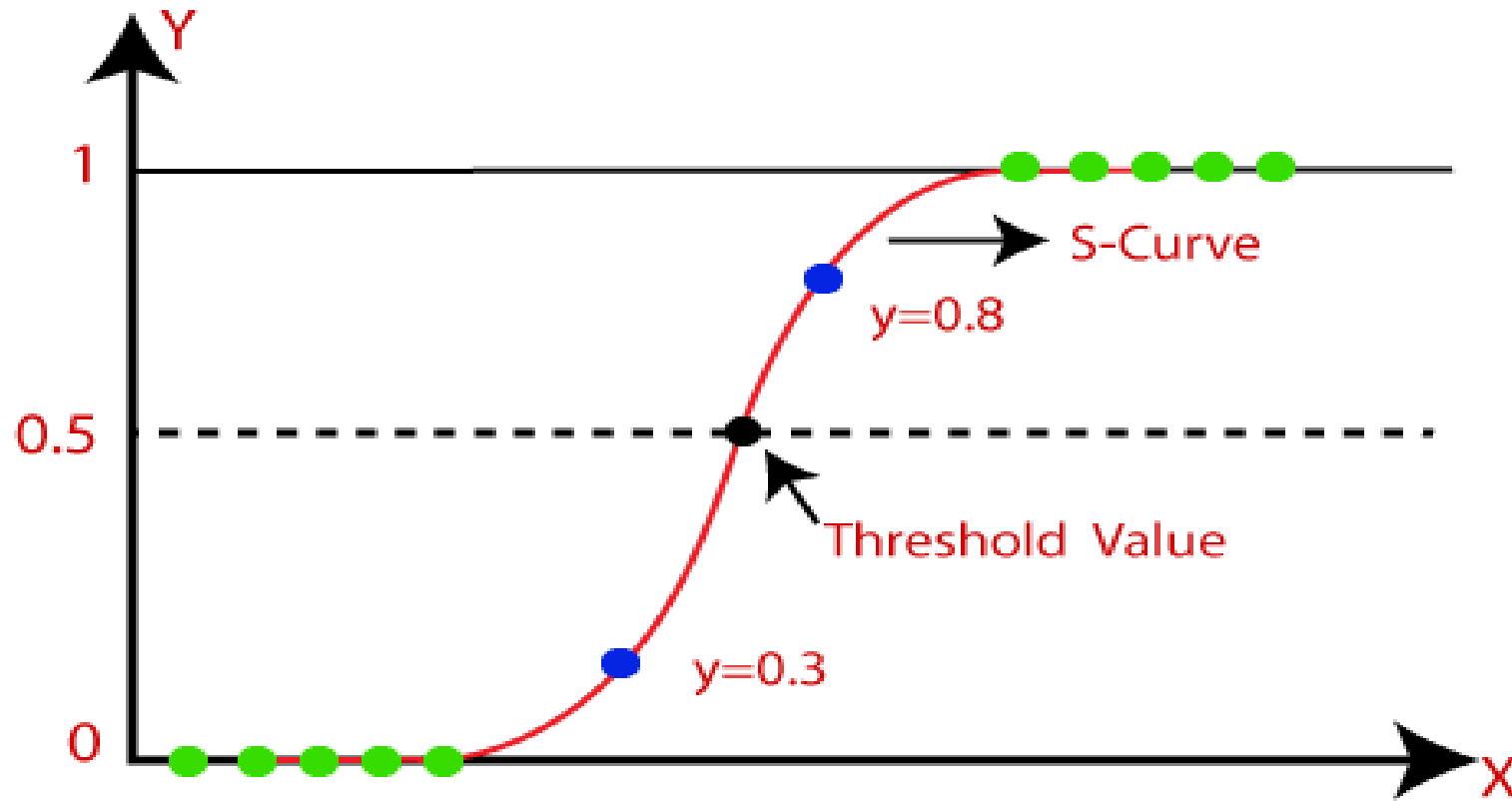


Logistic Regression in Machine Learning

- In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- The curve from the logistic function indicates the likelihood of something such as whether the cells are cancerous or not, a mouse is obese or not based on its weight, etc.
- Logistic Regression is a significant machine learning algorithm because it has the ability to provide probabilities and classify new data using continuous and discrete datasets.
- Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification. The below image is showing the logistic function:



Logistic Regression in Machine Learning





Logistic Function (Sigmoid Function):

- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1.
- The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form. The S-form curve is called the Sigmoid function or the logistic function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.



Assumptions for Logistic Regression:

- The dependent variable must be categorical in nature.
- The independent variable should not have multi-collinearity.



Logistic Regression Equation:

The Logistic regression equation can be obtained from the Linear Regression equation. The mathematical steps to get Logistic Regression equations are given below:

- We know the equation of the straight line can be written as:

$$y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

- In Logistic Regression y can be between 0 and 1 only, so for this let's divide the above equation by $(1-y)$:

$$\frac{y}{1-y} ; 0 \text{ for } y=0, \text{ and infinity for } y=1$$



Logistic Regression Equation:

- But we need range between $-\infty$ to $+\infty$, then take logarithm of the equation it will become:

$$\log \left[\frac{y}{1-y} \right] = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

- The above equation is the final equation for Logistic Regression.



K-Nearest Neighbor(KNN) Algorithm for Machine Learning

- K-Nearest Neighbors is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.



K-Nearest Neighbor(KNN) Algorithm for Machine Learning

- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

K-Nearest Neighbor(KNN) Algorithm for Machine Learning

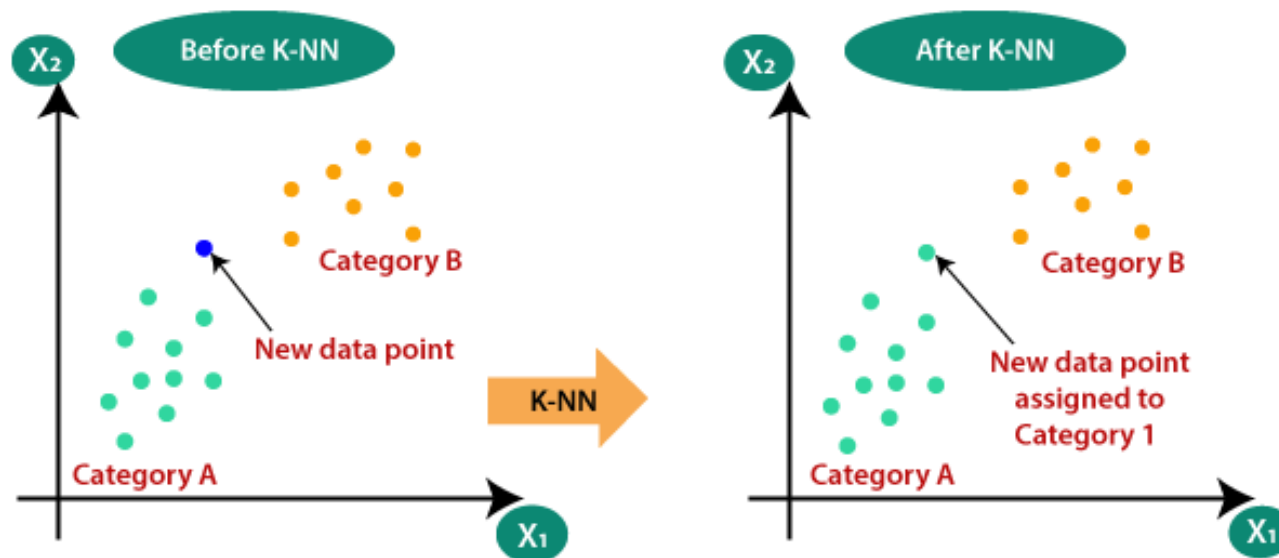
•**Example:** Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.





Why do we need a K-NN Algorithm?

- Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x_1 , so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:





How does K-NN work?

The K-NN working can be explained on the basis of the below algorithm:

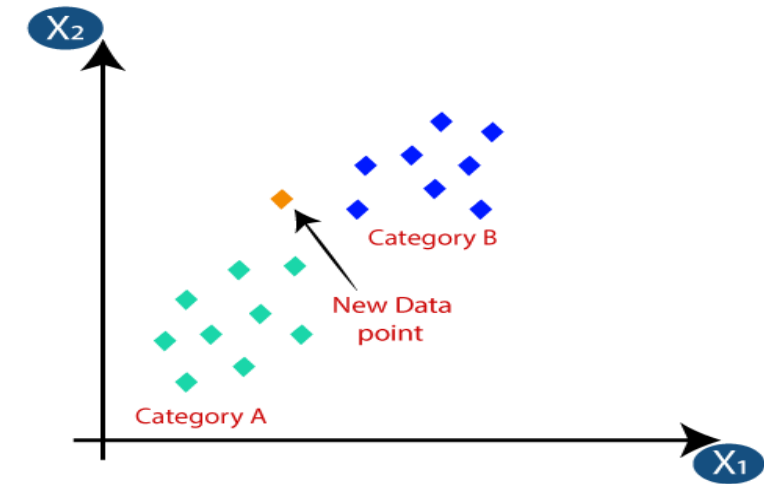
- Step-1:** Select the number K of the neighbors
- Step-2:** Calculate the Euclidean distance of **K number of neighbors**
- Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- Step-4:** Among these k neighbors, count the number of the data points in each category.
- Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- Step-6:** Our model is ready.



How does K-NN work?

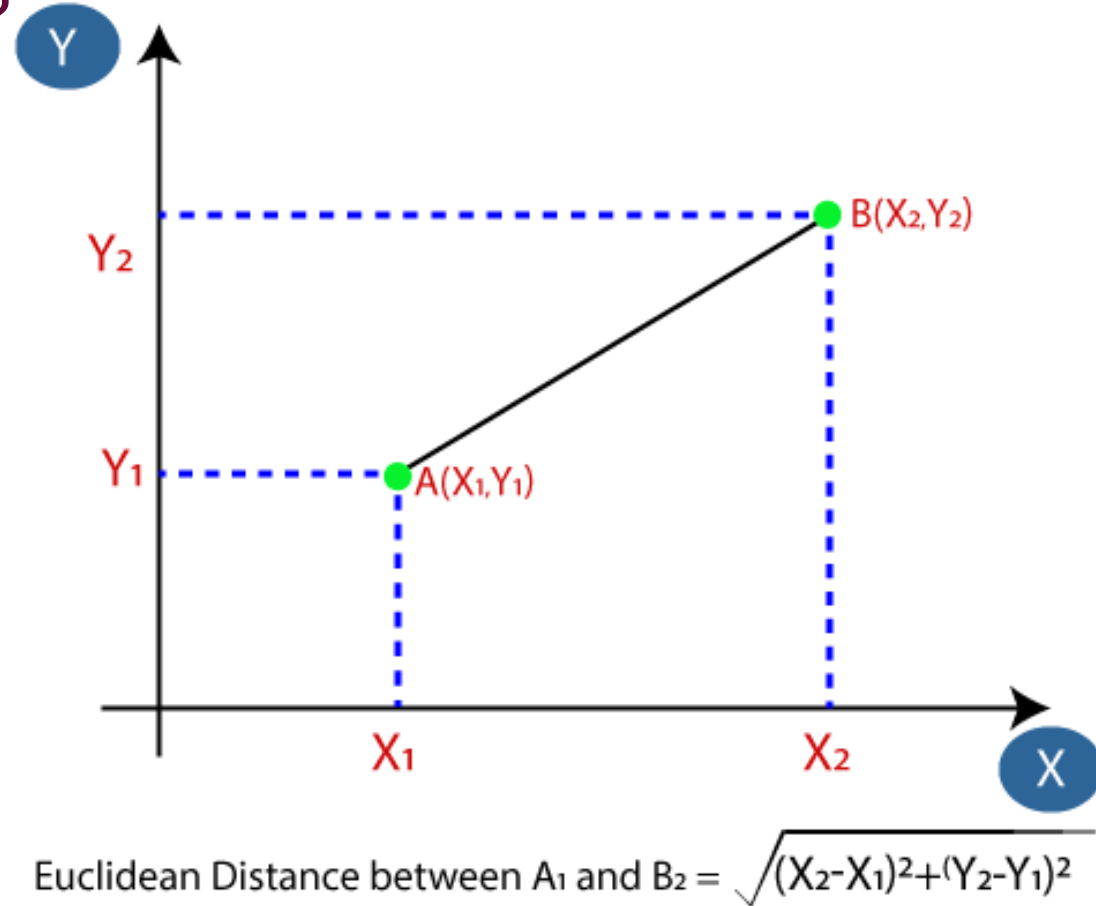
Suppose we have a new data point and we need to put it in the required category. Consider the below image:

- Firstly, we will choose the number of neighbors, so we will choose the $k=5$.
- Next, we will calculate the **Euclidean distance** between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry. It can be calculated as:





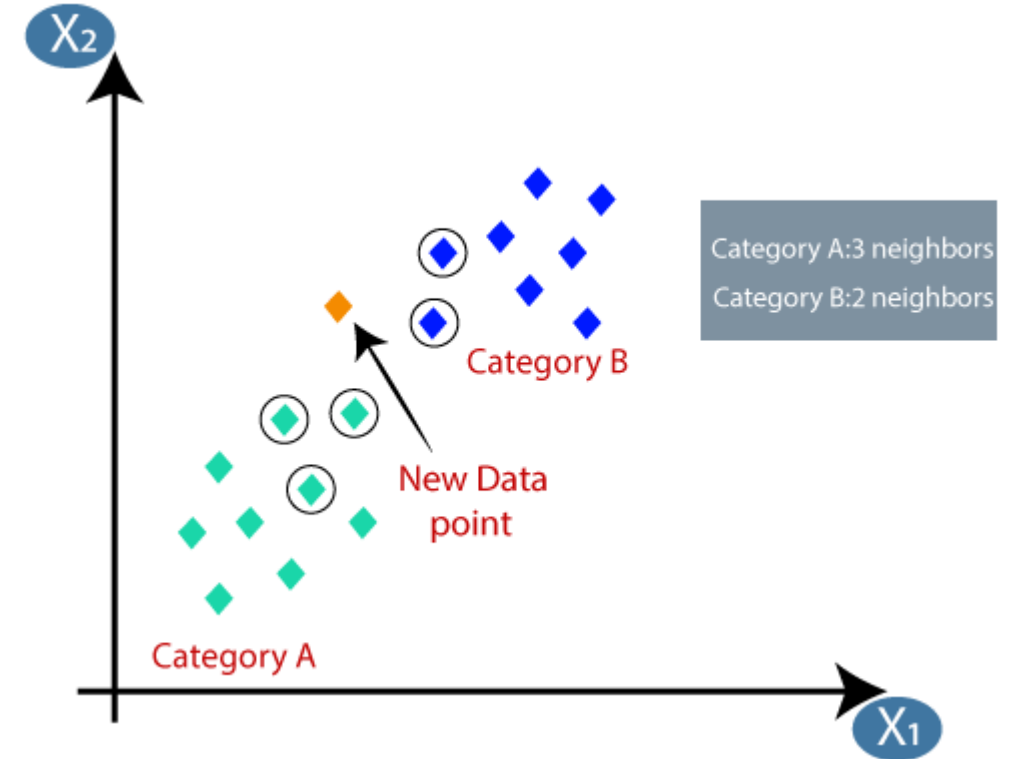
How does K-NN work?





How does K-NN work?

By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:





How to select the value of K in the K-NN Algorithm?

Below are some points to remember while selecting the value of K in the K-NN algorithm:

- There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.
- A very low value for K such as $K=1$ or $K=2$, can be noisy and lead to the effects of outliers in the model.
- Large values for K are good, but it may find some difficulties



Advantages of KNN Algorithm:

- It is simple to implement.
- It is robust to the noisy training data
- It can be more effective if the training data is large.



Disadvantages of KNN Algorithm:

- Always needs to determine the value of K which may be complex some time.
- The computation cost is high because of calculating the distance between the data points for all the training samples.

Example

Suppose we have the following data:

X1	X2	Y (label) (Target) (output)
4	3	1
1	3	1
3	3	1
3	7	1
7	4	1
4	1	1
6	5	1
5	6	1
3	6	1
6	7	1
4	6	0
4	4	0
5	8	0
7	8	0
5	6	0
10	5	0
7	6	0
4	10	0
9	7	0
5	4	0
8	5	0
6	6	0
7	4	0
8	8	0
6	5	??
2	1	??

If we determine $K = 8$ (we will use 8 nearest neighbors) as parameter of this algorithm. Then we calculate the distance between the query-instance and all the training samples. By use Euclidean distance, the distance between two points q and p is:

$$d(\mathbf{p}, \mathbf{q}) = d(\mathbf{q}, \mathbf{p}) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \cdots + (q_n - p_n)^2} = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}.$$

The next step is to find the K -nearest neighbors. We include a training sample as nearest neighbors if the distance of this training sample to the query instance is less than or equal to the K -th smallest distance. In other words, we sort the distance of all training samples to the query instance and determine the K -th minimum distance. The KNN prediction of the query instance is based on simple majority of the category of nearest neighbors. In our example, the category is only binary, thus the majority can be taken as simple as counting the number of '1' and '0'. If the number of '1's is greater than '0's, we predict the query instance as '1' and vice versa. If the number of '1' is equal to '0', we can choose any category.

X1	X2	Distance	y
4	3	$(4-6)^2 + (3-5)^2 = 8$	1
1	3	29	1
3	3	13	1
3	7	13	1
7	4	2	1
4	1	20	1
6	5	0	1
5	6	2	1
3	6	13	1
6	7	9	1
4	6	5	0
4	4	5	0
5	8	10	0
7	8	10	0
5	6	2	0
10	5	16	0
7	6	2	0
4	10	29	0
9	7	13	0
5	4	2	0
8	5	4	0
6	6	1	0
7	4	2	0
8	8	13	0
6	5		0
2	1		1



Support Vector Machine Algorithm

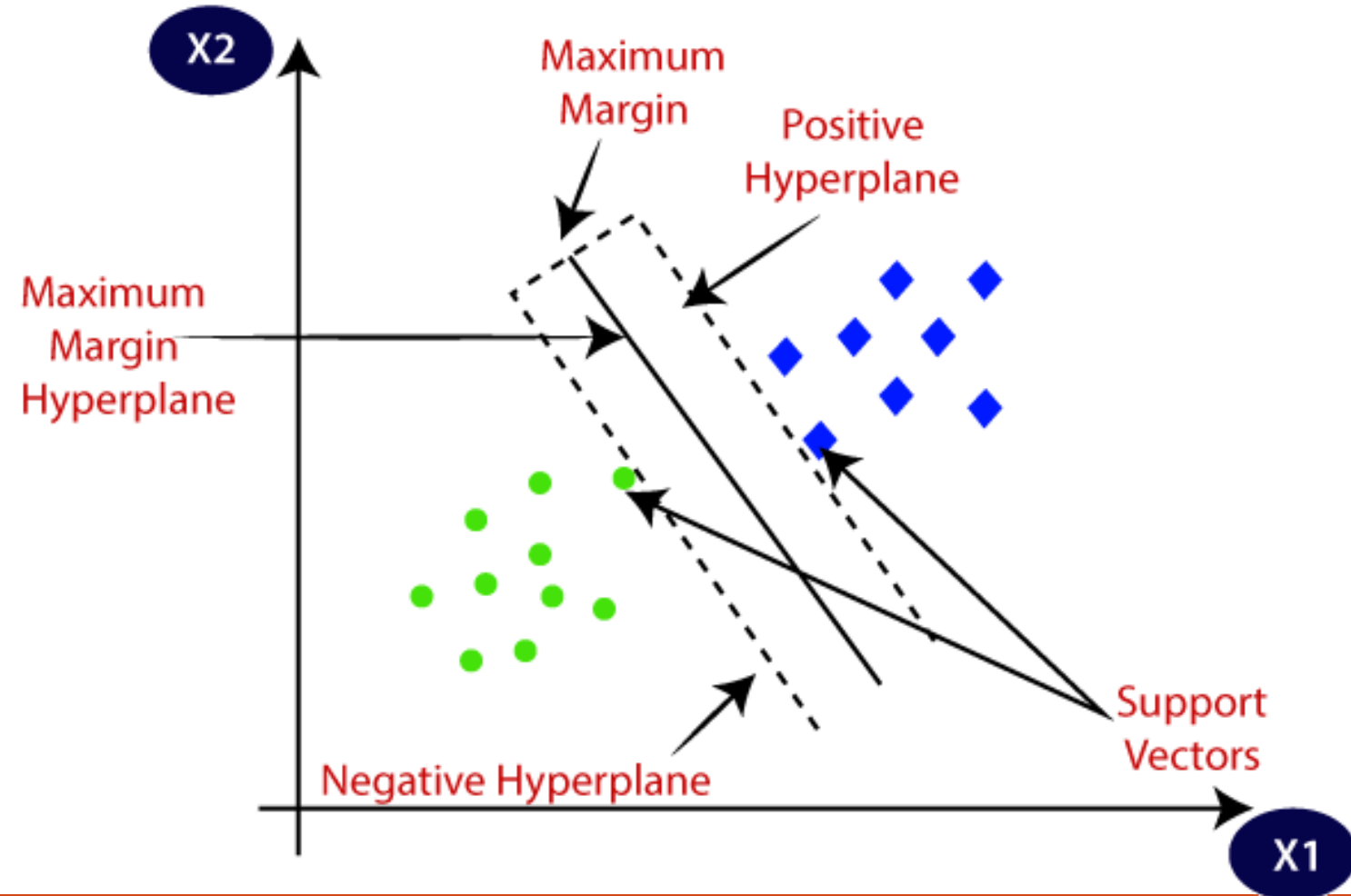
Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Support Vector Machine Algorithm



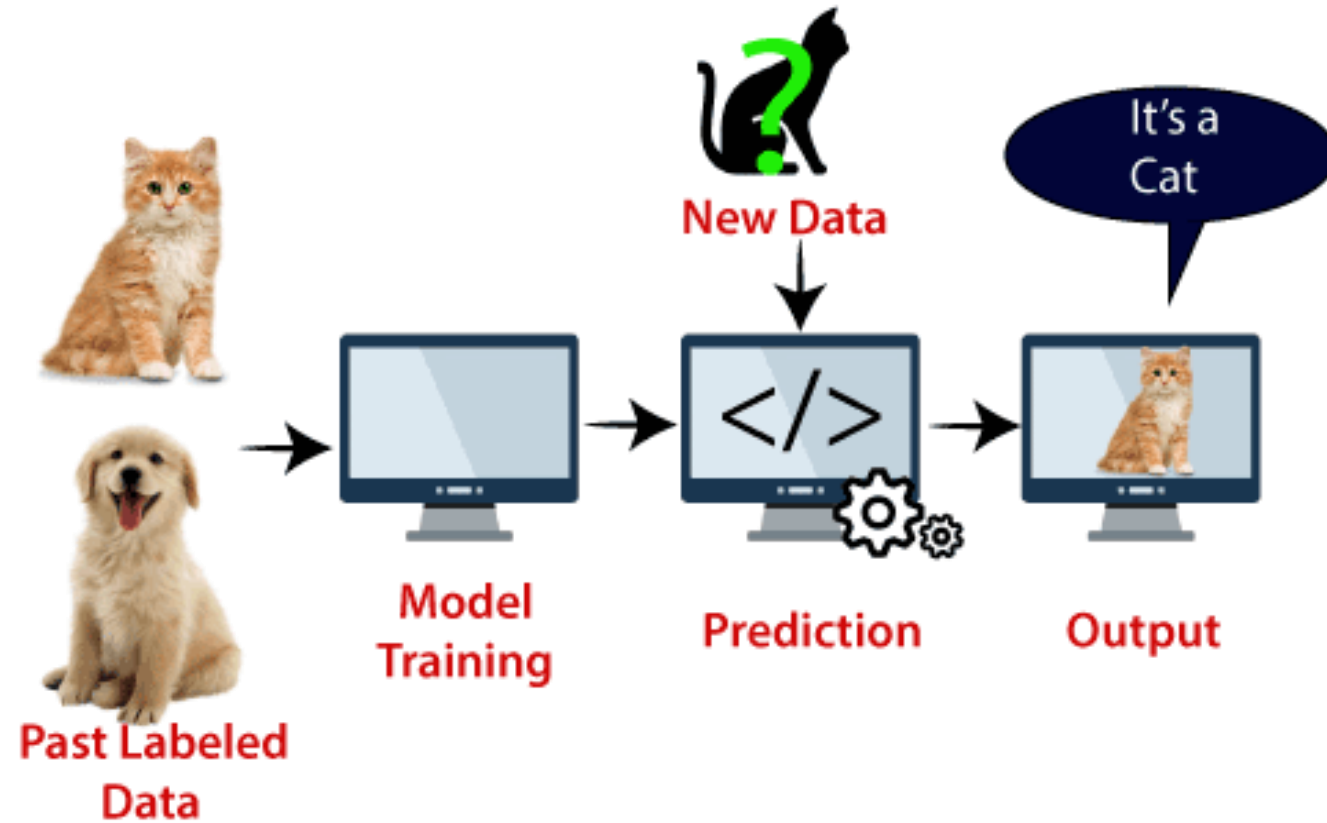


Support Vector Machine Algorithm

Suppose we see a strange cat that also has some features of dogs, so if we want a model that can accurately identify whether it is a cat or dog, so such a model can be created by using the SVM algorithm. We will first train our model with lots of images of cats and dogs so that it can learn about different features of cats and dogs, and then we test it with this strange creature. So as support vector creates a decision boundary between these two data (cat and dog) and choose extreme cases (support vectors), it will see the extreme case of cat and dog. On the basis of the support vectors, it will classify it as a cat. Consider the below diagram:



Support Vector Machine Algorithm





Types of SVM

SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.



Hyperplane and Support Vectors in the SVM algorithm:

Hyperplane: There can be multiple lines/decision boundaries to segregate the classes in n -dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.

We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

Support Vectors:

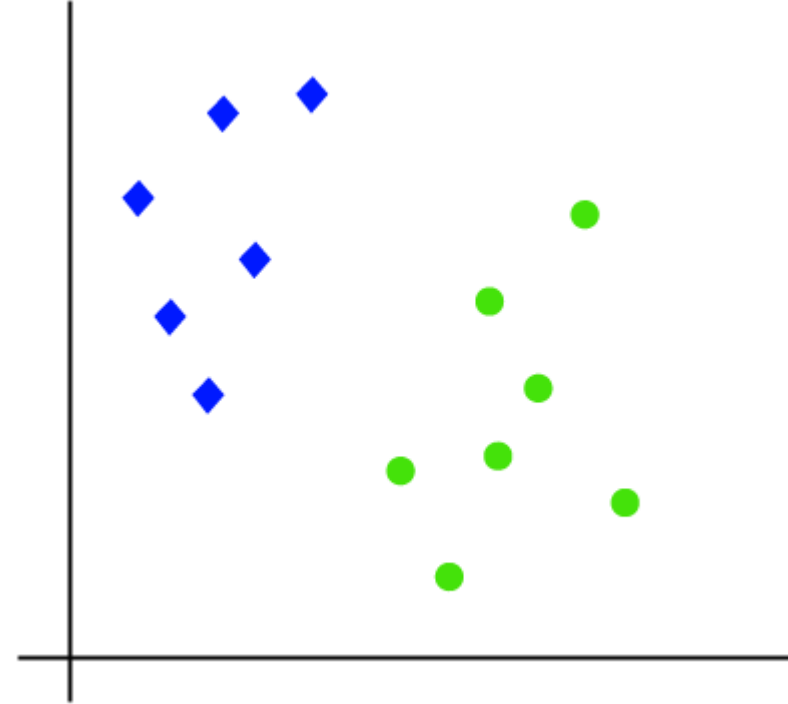
The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.



How does SVM works?

Linear SVM:

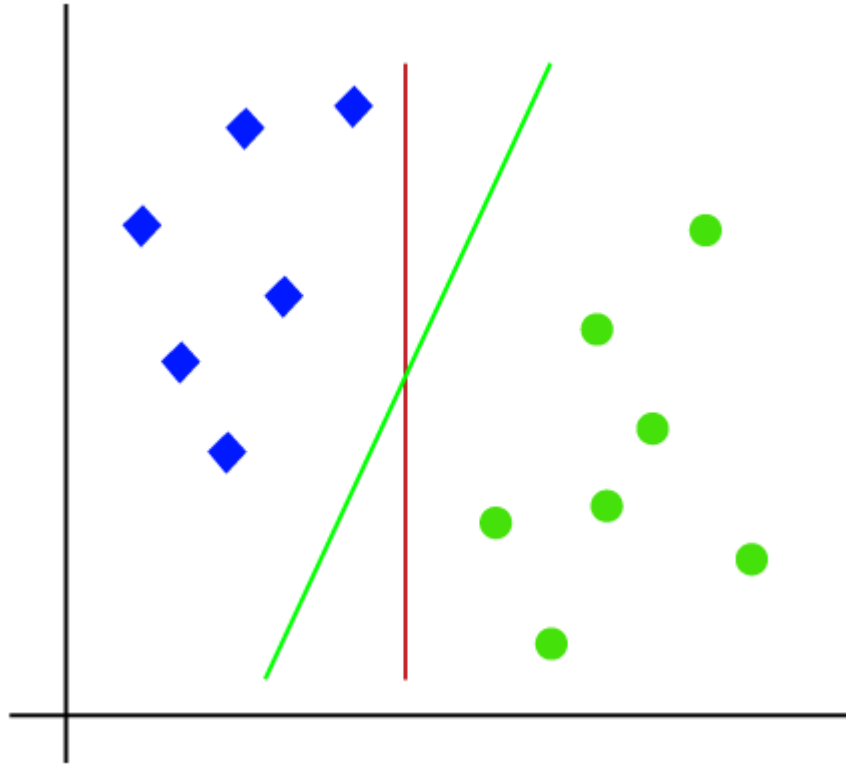
The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below image:





How does SVM works?

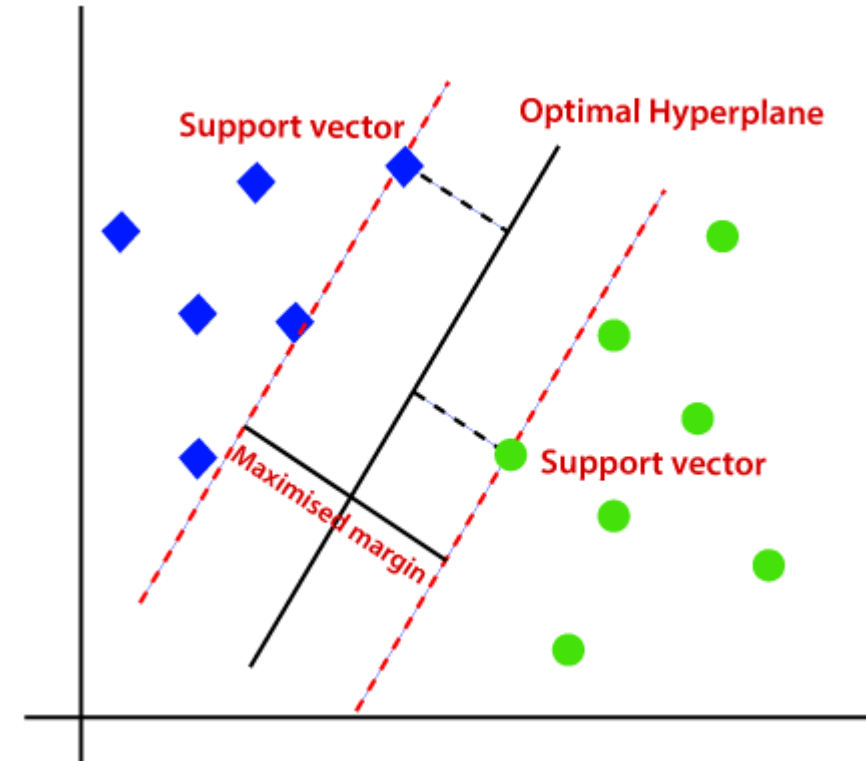
So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:





How does SVM works?

Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.

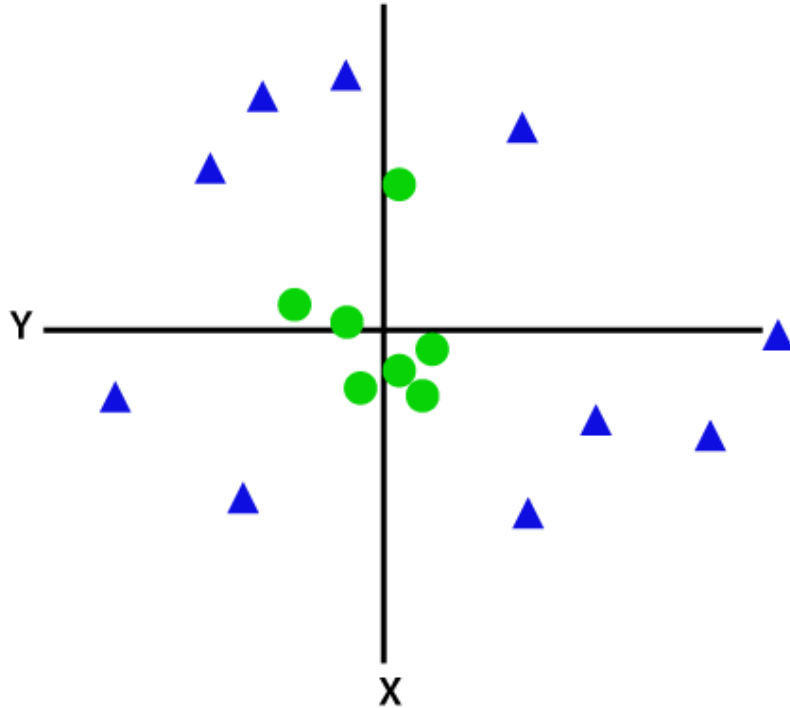




How does SVM works?

Non-Linear SVM:

If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:





How does SVM works?

So to separate these data points, we need to add one more dimension.

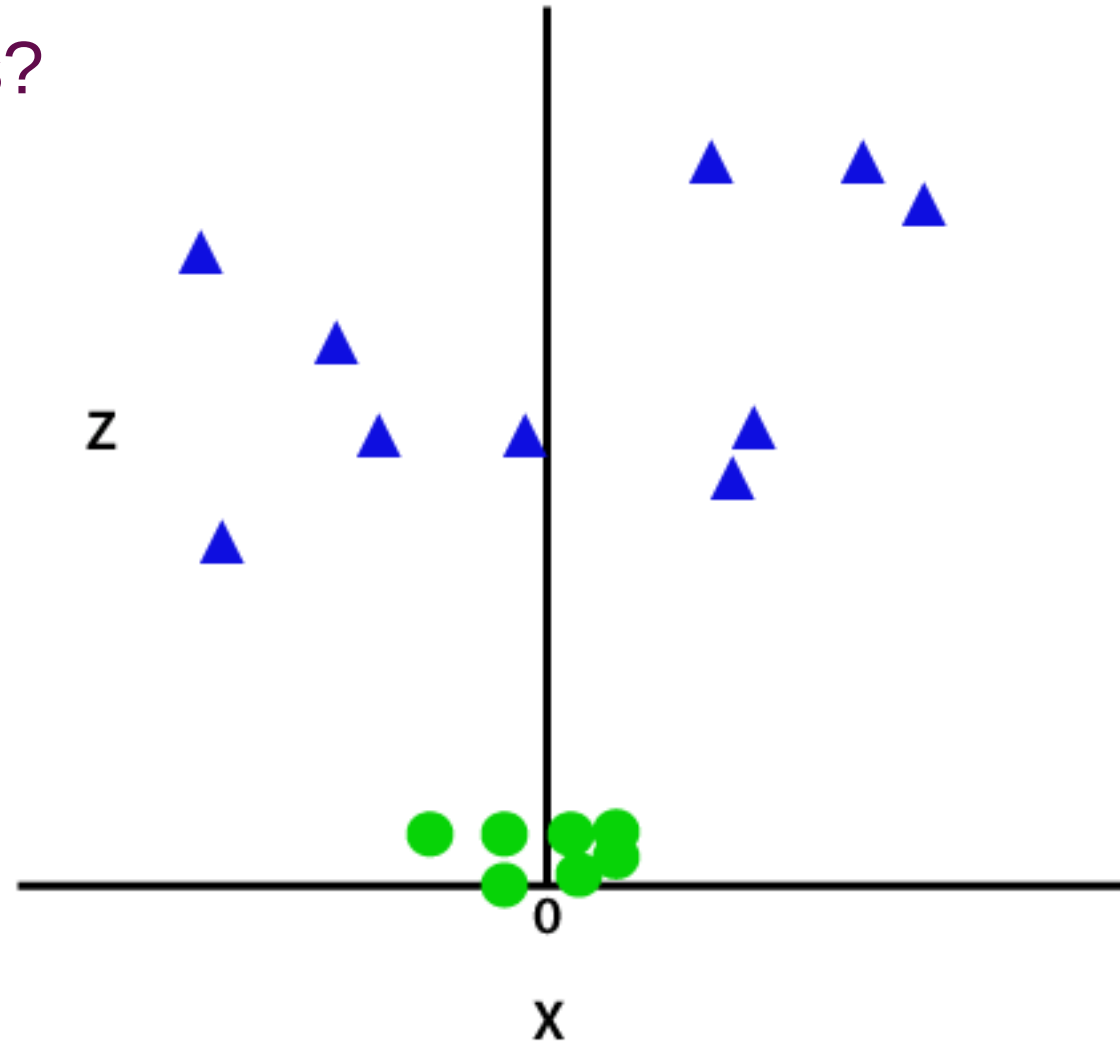
For linear data, we have used two dimensions x and y , so for non-linear data, we will add a third dimension z . It can be calculated as:

$$z = x^2 + y^2$$

By adding the third dimension, the sample space will become as below image:



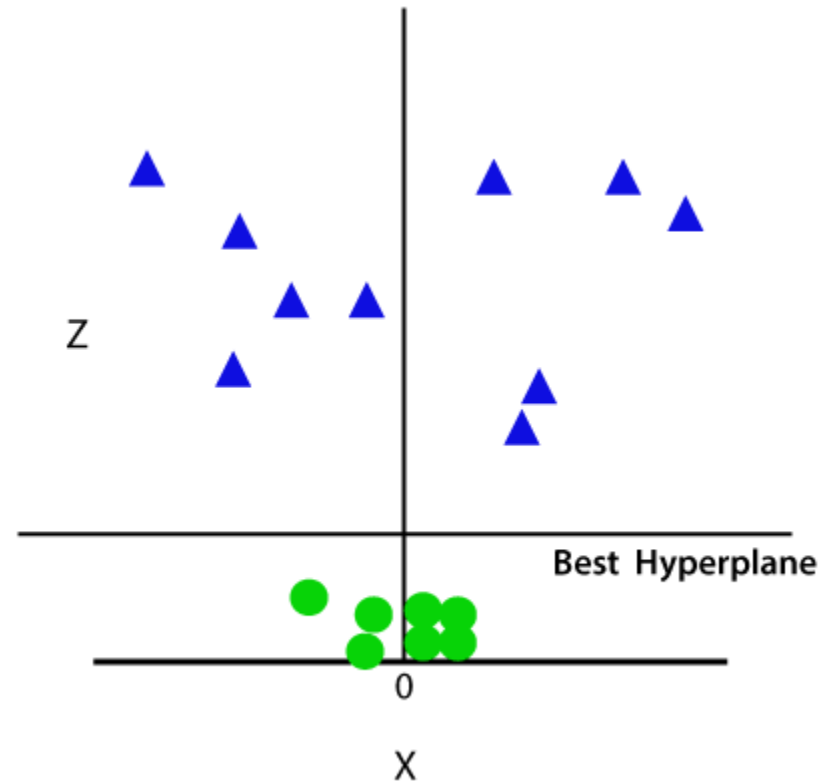
How does SVM works?





How does SVM works?

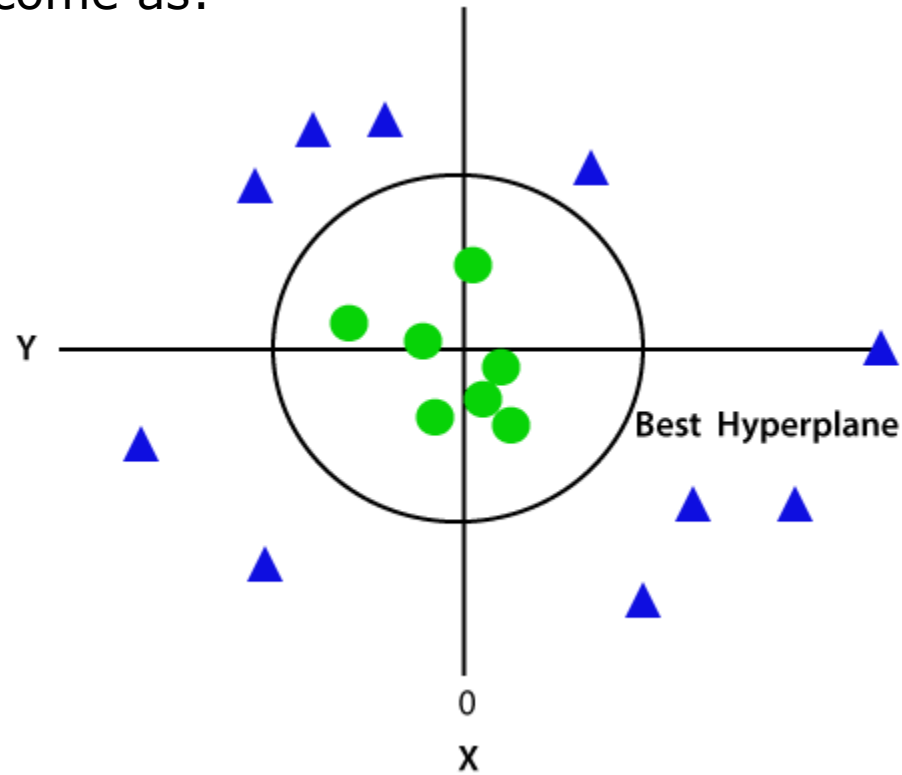
So now, SVM will divide the datasets into classes in the following way. Consider the below image:





How does SVM works?

Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with $z=1$, then it will become as:





Naive Bayes Classifier

Bayes Theorem:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Which tells us: how often *A* happens *given that B happens*, written **$P(A|B)$** ,

When we know: how often *B* happens *given that A happens*, written **$P(B|A)$**

and how likely *A* is on its own, written **$P(A)$**

and how likely *B* is on its own, written **$P(B)$**



Naive Bayes Classifier

Example:

- *Let us say $P(\text{Fire})$ means how often there is fire, and $P(\text{Smoke})$ means how often we see smoke, then:*
- $P(\text{Fire} | \text{Smoke})$ means how often there is fire when we can see smoke
- $P(\text{Smoke} | \text{Fire})$ means how often we can see smoke when there is fire



Naive Bayes Classifier

Example 1:

Example:

- dangerous fires are rare (1%)
- but smoke is fairly common (10%) due to barbecues,
- and 90% of dangerous fires make smoke

We can then discover the **probability of dangerous Fire when there is Smoke**:

$$\begin{aligned} P(\text{Fire}|\text{Smoke}) &= \frac{P(\text{Fire}) P(\text{Smoke}|\text{Fire})}{P(\text{Smoke})} \\ &= \frac{1\% \times 90\%}{10\%} \\ &= 9\% \end{aligned}$$

So it is still worth checking out any smoke to be sure.



Naive Bayes Classifier

Example 2:

- 1- The event of having eggs for breakfast is A $\rightarrow P(A) = 0.6$
- 2- The event of having Pizza for lunch is B $\rightarrow P(B) = 0.5$
- 3- $P(A|B) = 0.7$

Find $P(B|A)$?

$$P(B|A) = P(A|B) * P(B) / p(A) = 0.7 * 0.5 / 0.6 = 0.5833$$



Naive Bayes Classifier

Example 3:

You might be interested in finding out a patient's probability of having high blood pressure if they do not exercise.

- Let **A** means the event "Patient has high blood pressure." Past data tells you that 10% of patients entering your clinic have high blood pressure. **$P(A) = 0.10$** .
- Let **B** means that: "Patient does not exercise". Five percent of the clinic's patients does not exercise. **$P(B) = 0.05$** .
- You know that among those patients diagnosed with blood pressure, 7% does not exercise. This is your **$B|A$** : the probability that a patient does not exercise, given that they have high blood pressure, is 7%.

Bayes' theorem tells you: $P(A|B) = (0.07 * 0.1)/0.05 = 0.14$

- In other words, if the patient does not exercise, the chances of having high blood pressure is 0.14 (14%).



Naive Bayes Classifier

Day	Outlook	Temp.	Humidity	Wind	Decision
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Target
Predictors



Naive Bayes Classifier

Now, with regards to our dataset, we can apply Bayes' theorem in following way:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

where, y is class variable and X is a dependent feature vector (of size n) where:

$$X = (x_1, x_2, x_3, \dots, x_n)$$



Naive Bayes Classifier

$$P(y|x_1, \dots, x_n) = \frac{P(x_1|y)P(x_2|y)\dots P(x_n|y)P(y)}{P(x_1)P(x_2)\dots P(x_n)}$$

which can be expressed as:

$$P(y|x_1, \dots, x_n) = \frac{P(y) \prod_{i=1}^n P(x_i|y)}{P(x_1)P(x_2)\dots P(x_n)}$$

Now, as the denominator remains constant for a given input, we can remove that term:

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

Now, we need to create a classifier model. For this, we find the probability of given set of inputs for all possible values of the class variable y and pick up the output with maximum probability. This can be expressed mathematically as:

$$y = \operatorname{argmax}_y P(y) \prod_{i=1}^n P(x_i|y)$$



Naive Bayes Classifier

- We need to calculate $P(x_i | y_j)$ for each x_i in X and y_j in y manually.

Outlook				
	Yes	No	P(yes)	P(no)
Sunny	2	3	2/9	3/5
Overcast	4	0	4/9	0/5
Rainy	3	2	3/9	2/5
Total	9	5	100%	100%

Temperature				
	Yes	No	P(yes)	P(no)
Hot	2	2	2/9	2/5
Mild	4	2	4/9	2/5
Cool	3	1	3/9	1/5
Total	9	5	100%	100%

- For example, probability of playing golf given that the temperature is cool, i.e $P(\text{temp.} = \text{cool} | \text{play golf} = \text{Yes}) = 3/9$.



Naive Bayes Classifier

Humidity

	Yes	No	P(yes)	P(no)
High	3	4	3/9	4/5
Normal	6	1	6/9	1/5
Total	9	5	100%	100%

Wind

	Yes	No	P(yes)	P(no)
False	6	2	6/9	2/5
True	3	3	3/9	3/5
Total	9	5	100%	100%



Naive Bayes Classifier

- Next, we need to find class probabilities ($P(y)$). For example, $P(\text{play golf} = \text{Yes}) = 9/14$.

Play		$P(\text{Yes})/P(\text{No})$
Yes	9	9/14
No	5	5/14
Total	14	100%



Naive Bayes Classifier

Example: Find the decision for the following input
today = (Sunny, Hot, Normal, Weak)

$$P(Yes|today) = \frac{P(SunnyOutlook|Yes)P(HotTemperature|Yes)P(NormalHumidity|Yes)P(NoWind|Yes)P(Yes)}{P(today)}$$

and probability to not play golf is given by:

$$P(No|today) = \frac{P(SunnyOutlook|No)P(HotTemperature|No)P(NormalHumidity|No)P(NoWind|No)P(No)}{P(today)}$$

Since, $P(today)$ is common in both probabilities, we can ignore $P(today)$ and find proportional probabilities as:

$$P(Yes|today) \propto \frac{2}{9} \cdot \frac{2}{9} \cdot \frac{6}{9} \cdot \frac{6}{9} \cdot \frac{9}{14} \approx 0.0141$$

and

$$P(No|today) \propto \frac{3}{5} \cdot \frac{2}{5} \cdot \frac{1}{5} \cdot \frac{2}{5} \cdot \frac{5}{14} \approx 0.0068$$

Now, since

$$P(Yes|today) + P(No|today) = 1$$



Naive Bayes Classifier

Normalization:

$$P(Yes|today) = \frac{0.0141}{0.0141+0.0068} = 0.67$$

and

$$P(No|today) = \frac{0.0068}{0.0141+0.0068} = 0.33$$

Since

$$P(Yes|today) > P(No|today)$$

So, prediction that football would be played is: **Yes.**



Naive Bayes Classifier

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import datasets
```

```
iris = datasets.load_iris()
print(iris.feature_names)
```

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
print(iris.data[0:4])
```

```
→ [[5.1 3.5 1.4 0.2]
    [4.9 3. 1.4 0.2]
    [4.7 3.2 1.3 0.2]
    [4.6 3.1 1.5 0.2]]
```

```
print(iris.target_names)
```

```
→ ['setosa' 'versicolor' 'virginica']
```



Naive Bayes Classifier

```
# store the feature matrix (X) and response vector (y)
```

```
X = iris.data
```

```
y = iris.target
```

```
# splitting X and y into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4,  
random_state=1)
```

```
# training the model on training set
```

```
gnb = GaussianNB()
```

```
gnb.fit(X_train, y_train)
```

```
# making predictions on the testing set
```

```
y_pred = gnb.predict([[5.1,3.5,1.4,0.2]])
```

```
print(y_pred)
```

```
➔ [0]
```



Naive Bayes Classifier

Save Model Using Pickle

```
import pandas
```

```
from sklearn import model_selection
```

```
from sklearn.linear_model import LogisticRegression
```

```
import pickle
```

```
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
```

```
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
```

```
dataframe = pandas.read_csv(url, names=names)
```

```
array = dataframe.values
```

```
X = array[:,0:8]
```

```
Y = array[:,8]
```

```
X_train, X_test, Y_train, Y_test = model_selection.train_test_split(X, Y, test_size=0.3)
```



Naive Bayes Classifier

```
# Fit the model on training set
model = LogisticRegression()
model.fit(X_train, Y_train)
# save the model to disk
filename = 'finalized_model.sav'
pickle.dump(model, open(filename, 'wb'))

# some time later...

# load the model from disk
loaded_model = pickle.load(open(filename, 'rb'))
result = loaded_model.score(X_test, Y_test)
print(result)
```